# End of Day Report

| | |
|---|---|
| **Name** | Atibali Saiyed |
| **Date** | 09 November 2026 |
| **GitHub Repository** | https://github.com/Atibali/Gateway-Training/tree/main/Week2/Day1 |

## Part 1 — Web Fundamentals and HTTP

**Web fundamentals —** Covered the client–server model, browsers, servers, URLs, domains, and DNS resolution.

**HTTP —** Learned HTTP methods (GET, POST, PUT, PATCH, DELETE), status code families (2xx–5xx), headers, and request and response bodies.

**REST —** Studied resources and URLs, statelessness, idempotency, and paging and filtering through query strings.

**JSON —** Reviewed JSON syntax rules, objects and arrays, serialization, and parsing.

**Browser DevTools —** Explored the Elements, Console, and Network tabs.

**Storage and security —** Covered cookies, local storage, the same-origin policy, and the concept of CORS.

## Part 2 — Software Testing

**Black-box test design —** Learned equivalence partitioning, two-value and three-value boundary value analysis, decision tables, state transition testing, and use-case testing.

**Experience-based testing —** Covered error guessing, checklist-based testing, and an introduction to exploratory testing.

**Test artifacts —** Distinguished test scenarios from test cases, and reviewed positive and negative cases, preconditions, and test data.

**Defect management —** Learned defect life-cycle states and transitions.

**Severity versus priority —** Understood severity as the impact of a defect and priority as the urgency of fixing it; reviewed the severity–priority grid and worked examples.

**Bug reporting —** Learned to write clear titles and numbered reproduction steps, specify expected versus actual results, include environment and build details, and attach appropriate evidence.

## Key Learnings

- Understood the request–response cycle, URL anatomy, and how DNS maps domain names to IP addresses.
- Learned HTTP method semantics, including safe and idempotent methods, and how to interpret status codes.
- Grasped REST principles such as resource-oriented URLs, statelessness, and query-string paging and filtering.
- Practised JSON syntax and distinguished serializing data from parsing it.
- Used browser DevTools to inspect network requests, DOM elements, and console output.
- Distinguished cookies from local storage and understood that CORS is enforced by browsers.
- Learned to select suitable black-box testing techniques based on input ranges and business-rule complexity.
- Recognized the importance of positive and negative testing, clear preconditions, and appropriate test data.
- Mapped the defect life cycle and distinguished defect severity from fixing priority.
- Identified the elements of a clear, reproducible bug report supported by relevant evidence.

## Practical Activities

- Dissected URLs, performed DNS lookups, and ran a local HTTP server in the browser.
- Executed curl requests using HTTP methods and explored status-code responses.
- Explored a public REST API, comparing collection and item responses and paging behavior.
- Validated and parsed JSON in the browser console, including intentionally introduced syntax errors.

- Inspected live requests in the Network tab and debugged a failing fetch request.
- Tested cookies and local storage in DevTools; triggered and analyzed a CORS error and preflight request.
- Applied equivalence partitioning and boundary value analysis to sample input fields.
- Created decision tables and state transition diagrams, and derived test cases from use cases.
- Conducted a time-boxed exploratory testing session using a charter and debrief.
- Created test cases from scenarios and drafted bug reports using a standard template with screenshots, console logs, and HAR files as supporting evidence.

## Summary

Today I learned core web concepts, HTTP communication, REST API principles, JSON handling, browser DevTools, and browser storage and security concepts. I also strengthened my software testing knowledge by applying black-box test design techniques, experience-based testing, defect life-cycle concepts, severity and priority analysis, and structured bug reporting.

**Practical progress:** Completed hands-on exercises involving HTTP requests, REST APIs, JSON, DevTools, CORS, test design techniques, exploratory testing, and defect documentation.